

EdTech Platform — System Architecture Document

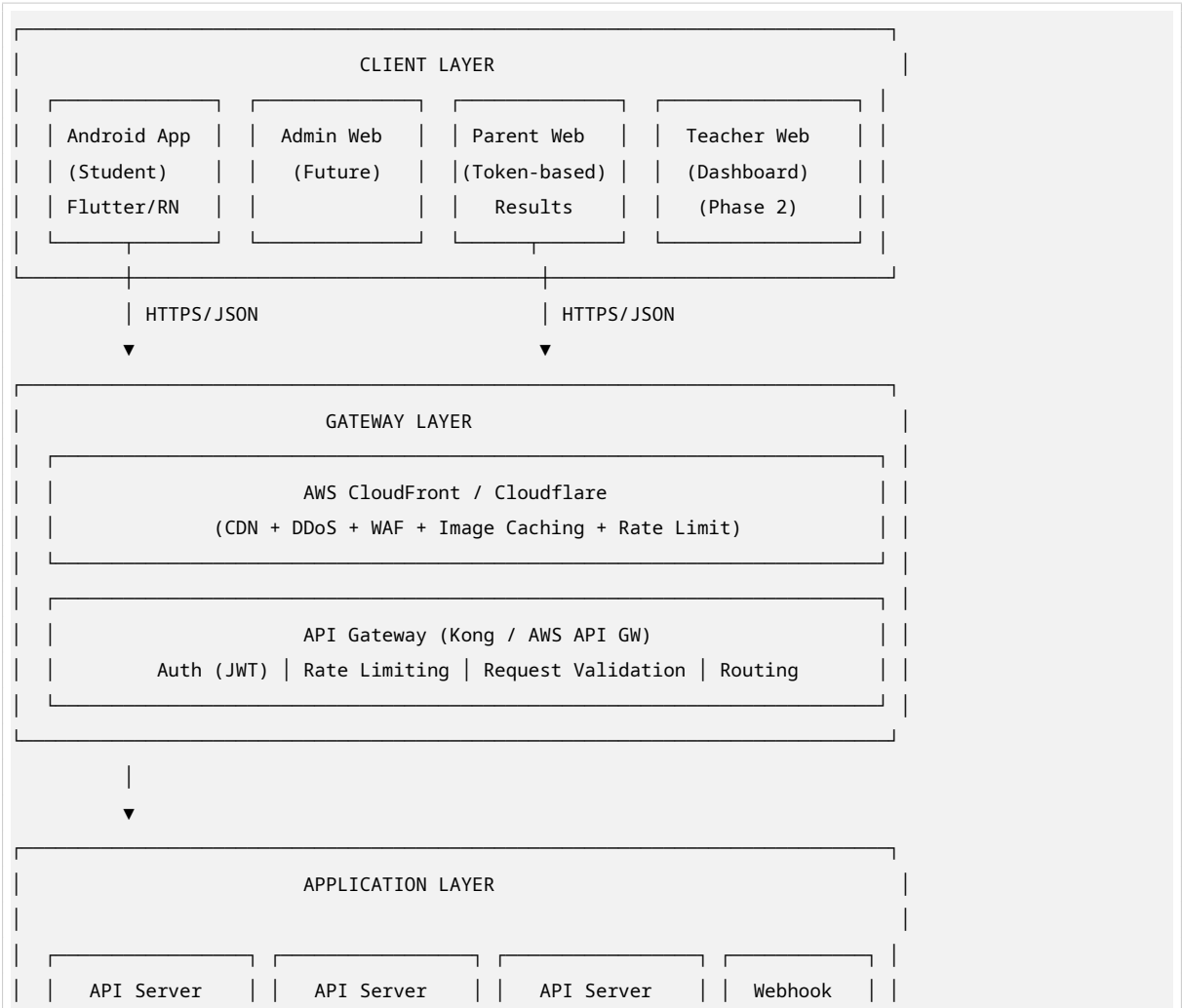
Version: 1.0 (Derived from Consolidated Spec v2.0)
Date: 2026-08-29
Status: Architecture Foundation
Target: MVP Phase 1 — Single-Teacher Coaching Centers (B2B)

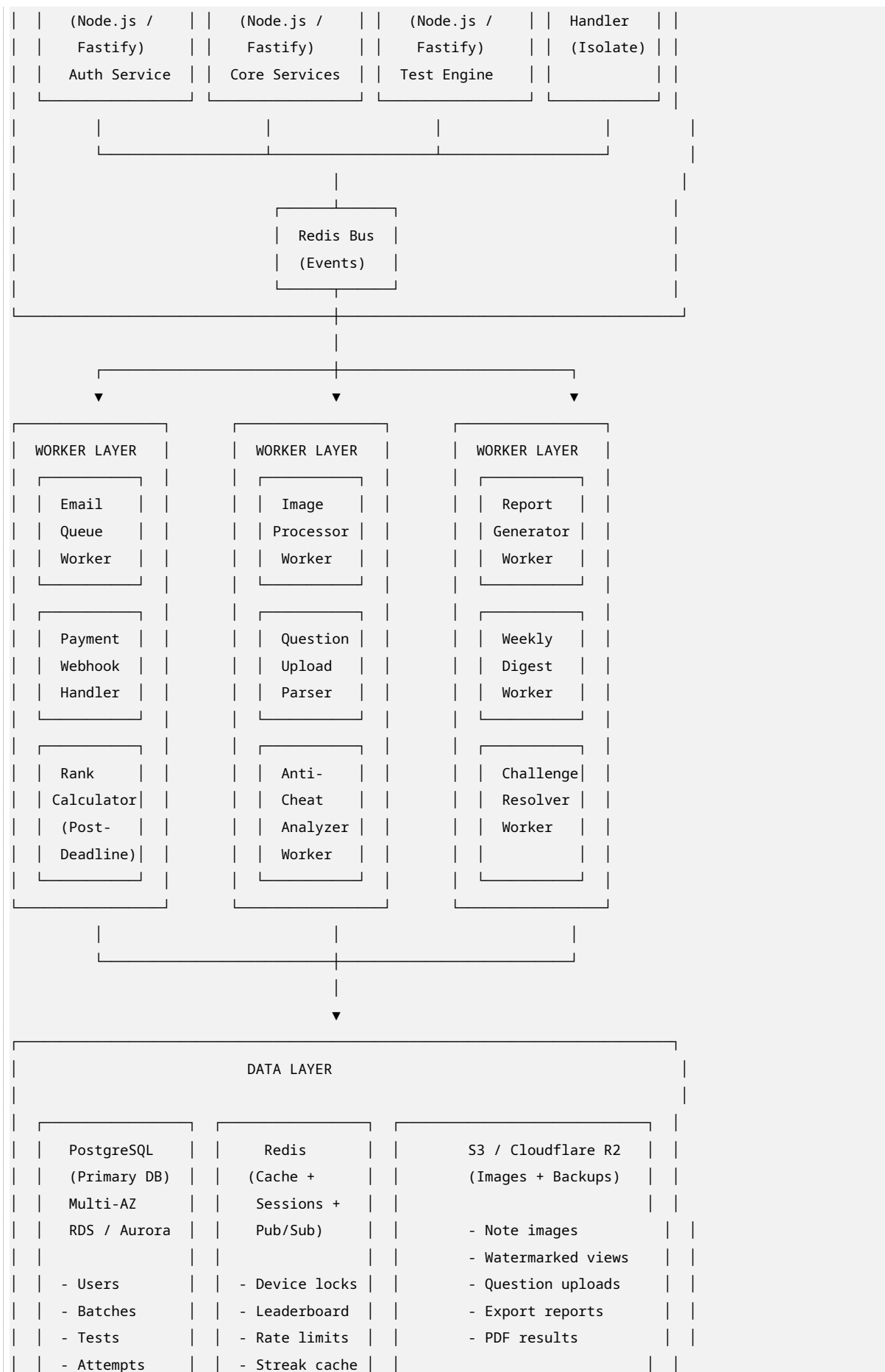
1. Executive Summary

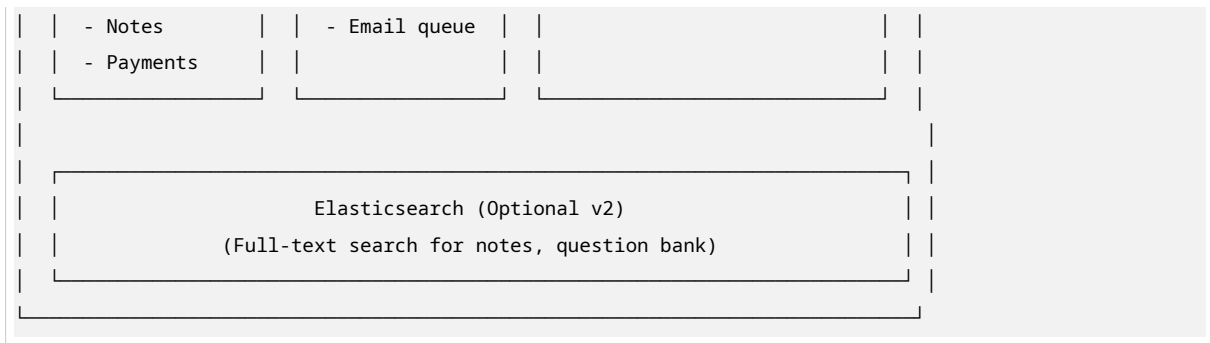
This document defines the system architecture for the EdTech platform serving single-teacher coaching centers (Classes 9–10, Math + Science, CBSE). The architecture prioritizes:

- **Reliability** over complexity (MVP-first)
- **Offline resilience** for test attempts and practice sessions
- **Cost efficiency** for small-center economics
- **Security** for anti-cheat and content protection
- **Scalability** to 10,000+ centers without re-architecture

2. High-Level Architecture







3. Service Decomposition

3.1 Core API Services

Service	Responsibility	Scaling Strategy
Auth Service	JWT issuance, OTP verification, session management, device fingerprinting	Horizontal, stateless
Student Service	Home screen data, profile, streaks, recent activity, bookmarks	Horizontal, cached
Teacher Service	Dashboard, batch management, student CRUD, onboarding state	Horizontal, stateless
Test Engine	Test creation, attempt lifecycle, auto-save, submission, anti-cheat	Horizontal, sticky sessions for active attempts
Practice Service	Chapter practice sessions, daily caps, monthly slots, retirement logic	Horizontal, cached
Content Service	Notes CRUD, image upload/compression, watermarking, view tracking	Horizontal, async image processing
Notification Service	In-app notifications, notice management, read receipts	Horizontal, stateless
Challenge Service	Challenge submission, admin queue, retroactive corrections	Horizontal, stateless
Admin Service	Question upload jobs, center settings, retirement config	Horizontal, stateless
Payment Service	Razorpay integration, subscription lifecycle, tier enforcement	Horizontal, webhook isolated
Email Service	Gmail OAuth, parent email queue, weekly digest, rate limit management	Horizontal, queue-backed

3.2 Worker Services

Worker	Trigger	Responsibility
Image Processor	S3 upload event	Compress thumbnails (50KB), full-view (500KB), generate watermarks

Table 2 – continued

Worker	Trigger	Responsibility
Question Parser	Upload job creation	Parse CSV/JSON, validate MCQ structure, detect duplicates, OCR for PDF
Rank Calculator	Test deadline + grace period	Calculate test ranks, update batch cumulative ranks, leaderboard refresh
Email Dispatcher	Cron (daily batches)	Send parent result emails, weekly digests, respect Gmail 100/day limit
Subscription Monitor	Cron (hourly)	Trial expiry, grace period enforcement, feature restriction, downgrade
Batch Janitor	Cron (daily)	Archive old batches, hard-delete after 1 year, cleanup inactive BatchStudent links
Streak Resetter	Cron (daily midnight IST)	Reset streaks for inactive students, monthly slot counters
Anti-Cheat Analyzer	Attempt submission	Aggregate tab switches, flag patterns, generate teacher alerts
Challenge Resolver	Admin action	Retroactively fix scores, recalculate ranks, notify affected students
Export Generator	Teacher request	Generate Excel/PDF reports (premium), async delivery

4. Database Schema Design

4.1 PostgreSQL (Primary)

Partitioning Strategy:

- `student_attempts`: Partition by `created_at` (monthly) — high write volume
- `practice_sessions`: Partition by `created_at` (monthly)
- `note_views`: Partition by `viewed_at` (monthly) — high cardinality
- `in_app_notifications`: Partition by `created_at` (monthly)

Indexing Strategy:

```
-- High-frequency lookups
CREATE INDEX idx_batch_student_active ON batch_students(batch_id, student_id, status);
CREATE INDEX idx_test_status_teacher ON tests(teacher_id, status, deadline);
CREATE INDEX idx_attempt_test_student ON student_attempts(test_id, student_id, status);
CREATE INDEX idx_practice_student_date ON practice_sessions(student_id, created_at DESC);

-- Notification & unread queries
CREATE INDEX idx_notes_batch_created ON notes(batch_id, created_at DESC) WHERE is_deleted = false;
CREATE INDEX idx_notice_batch_active ON notices(batch_id, expires_at) WHERE is_active = true;

-- Tier enforcement
CREATE INDEX idx_slot_counter_student_month ON practice_slot_counters(student_id, month);
```

```
-- Device locking
CREATE INDEX idx_device_session_attempt ON active_device_sessions(attempt_id, is_active);
```

Key Constraints:

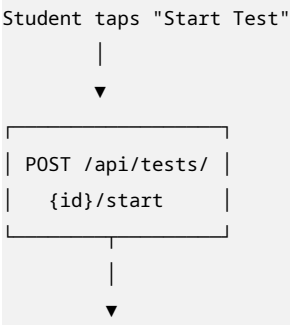
- UNIQUE (student_id, topic_id, date) on topic_attempt_caps (daily cap enforcement)
- UNIQUE (student_id, month) on practice_slot_counters (monthly limit)
- UNIQUE (note_id, student_id) on note_views (idempotency)
- UNIQUE (notice_id, student_id) on notice_reads

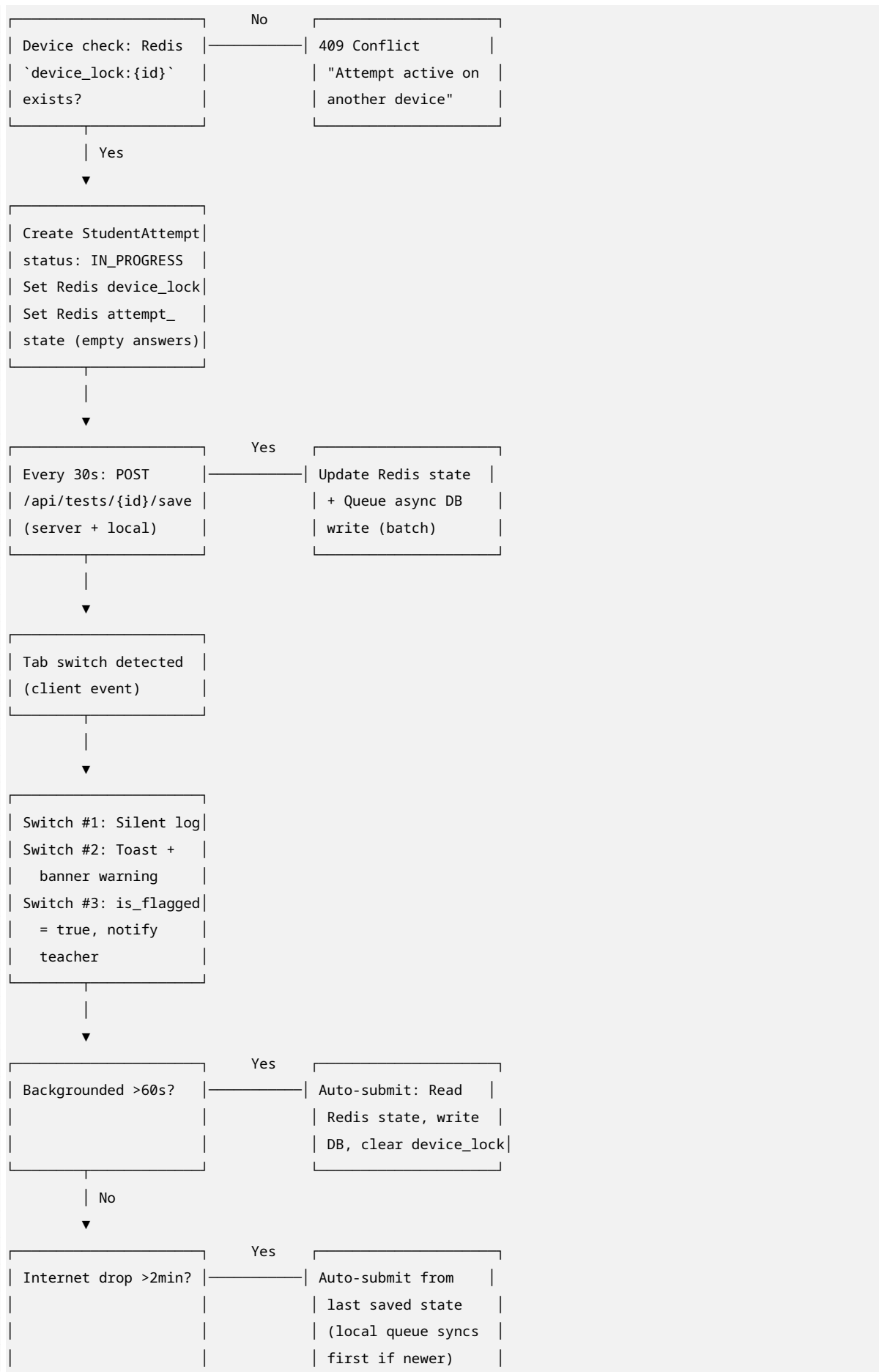
4.2 Redis Data Structures

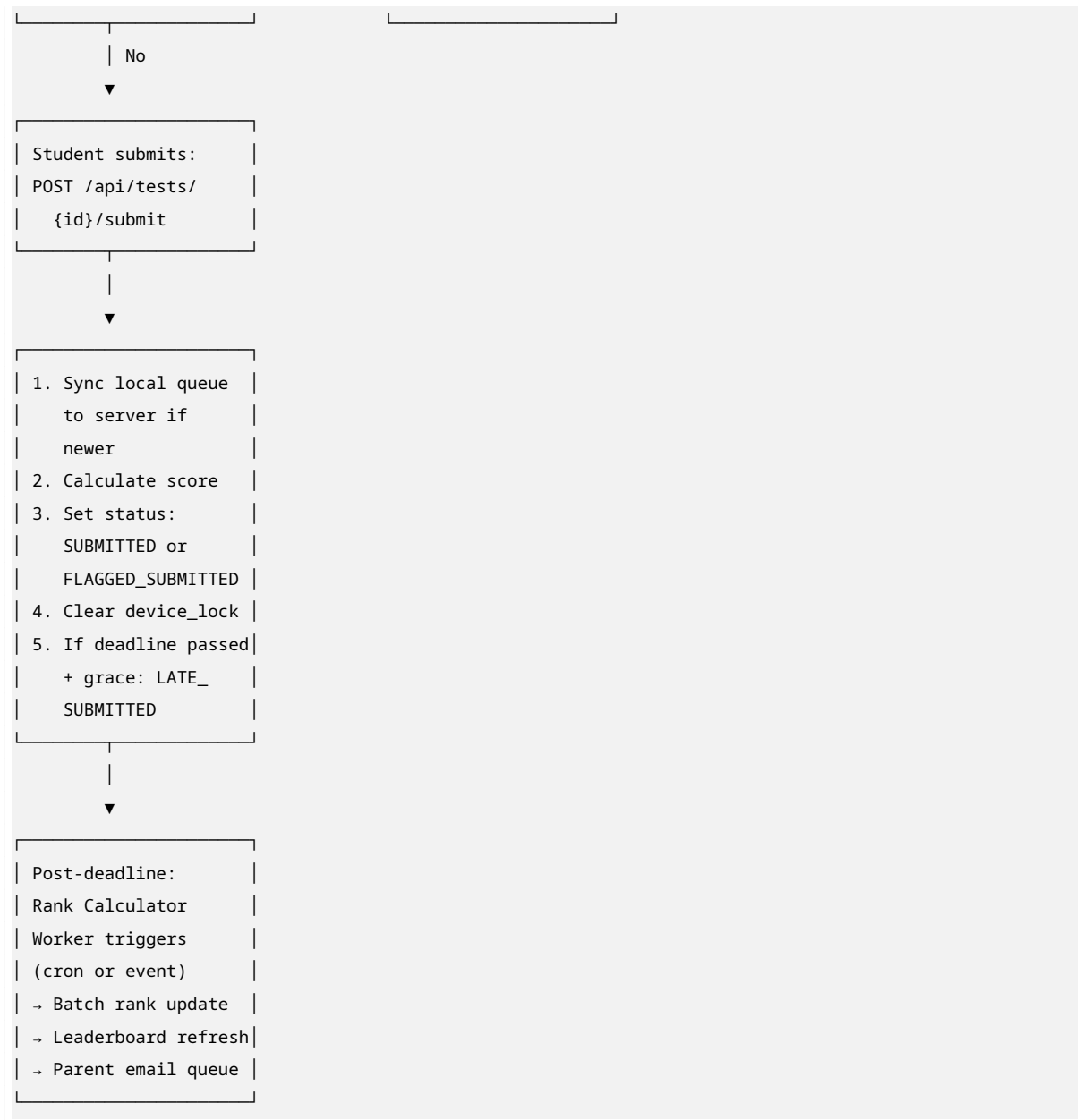
Key Pattern	Type	TTL	Purpose
device_lock:{attempt_id}	String	4h	Active device session fingerprint
attempt_state:{attempt_id}	Hash	4h	Latest auto-save answers (server + local sync)
leaderboard:{batch_id}	Sorted Set	7d	Cumulative rank scores (ZADD/ZRANK)
streak:{student_id}	Hash	24h	Current streak, longest, last practice date
notification_count:{user_id}:{type}	String	7d	Unread notification count per type
rate_limit:gmail:{teacher_id}	String	24h	Daily email counter (max 100)
daily_cap:{student_id}:{topic_id}:{date}	String	24h	Practice attempts today
test_preview:{test_id}	Hash	1h	Generated question set per student (randomization cache)
session:{jwt_id}	Hash	30d	JWT blacklist / refresh token rotation

5. Critical Data Flows

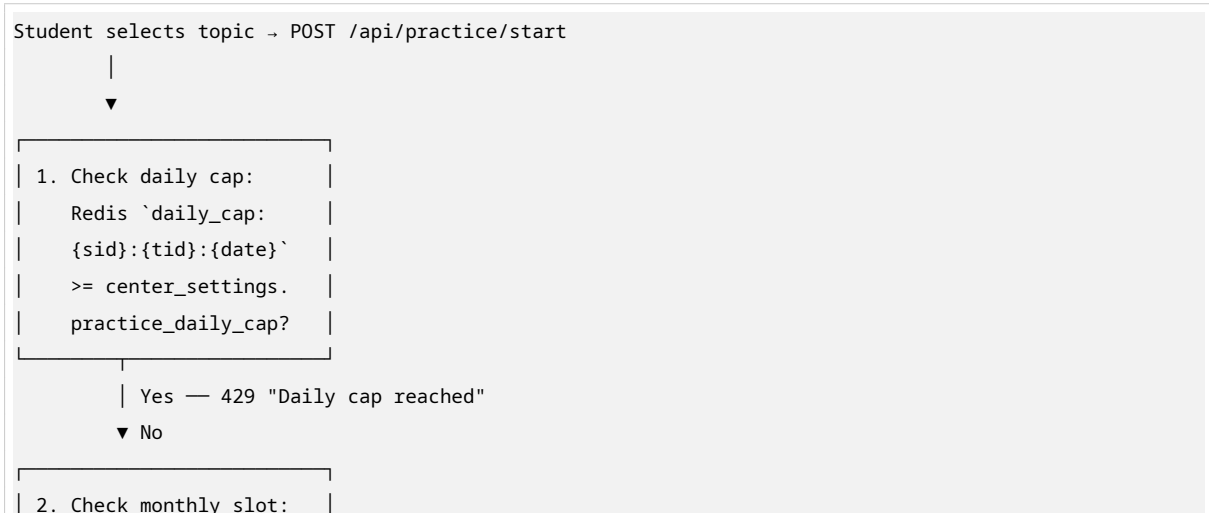
5.1 Test Attempt Lifecycle (with Anti-Cheat)





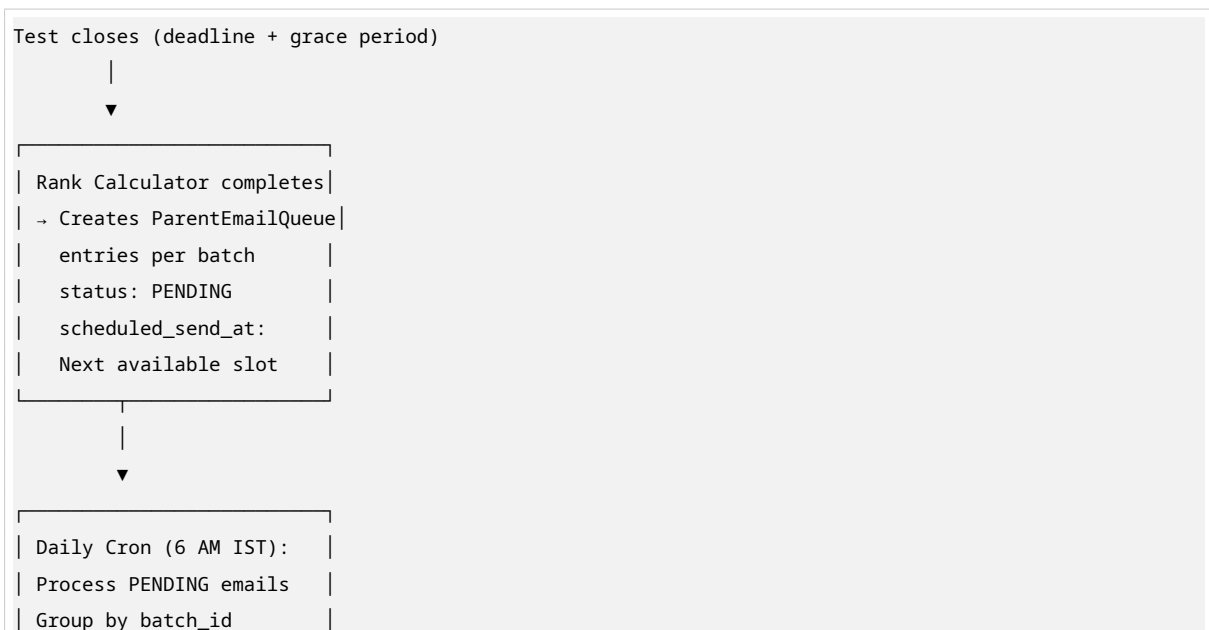


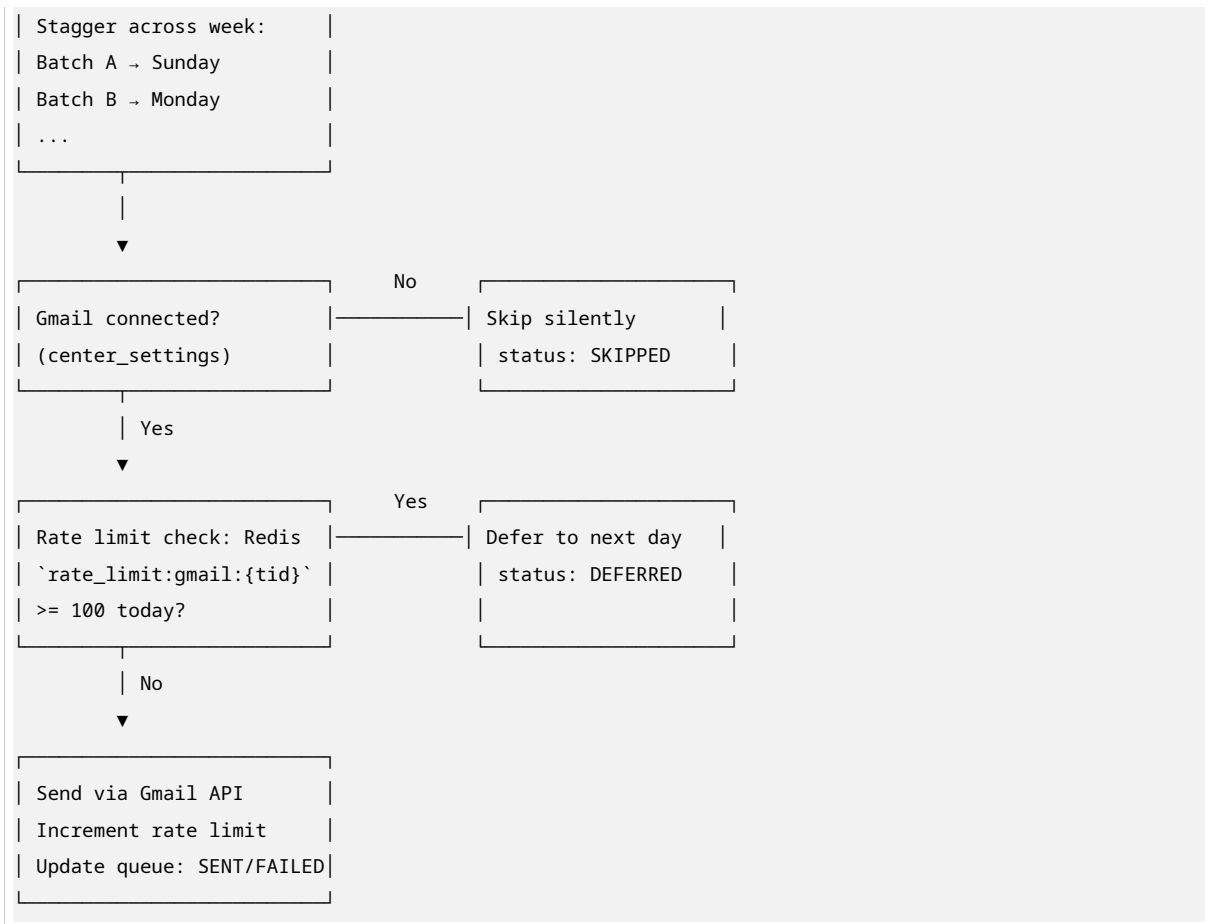
5.2 Chapter Practice Flow (with Tier Enforcement)





5.3 Parent Email Batch Flow (Gmail Rate Limit)





6. API Design Principles

6.1 Endpoint Patterns

/api/home	→ Composite aggregation (5 parallel DB queries)
/api/tests/{id}/start	→ Atomic device lock (Redis SET NX + DB transaction)
/api/tests/{id}/save	→ Fire-and-forget to Redis, async DB batch write
/api/tests/{id}/sync-local	→ Conflict resolution: server state vs. local queue
/api/practice/start	→ Multi-layer validation (cap → slot → retirement)
/api/notes/{id}/image/{image_id}	→ Signed URL generation (S3 presigned / Cloudflare)

6.2 Response Contracts

Home Screen (Composite):

```
{
  "student_name": "Aarav",
  "avg_score": 78.5,
  "batch_rank": 3,
  "current_streak": 7,
  "has_urgent_test": true,
  "urgent_test": {
    "id": "uuid",
    "title": "Class Test: Quadratic Equations",
```

```
    "deadline": "2026-08-29T18:00:00Z",
    "time_remaining": "4 hours left"
  },
  "notices": [{ "id": "uuid", "message": "...", "ttl_hours": 168 }],
  "unread_notes_count": 2,
  "recent_activities": [
    { "type": "TEST", "title": "Class Test: Circles", "score": "8/10", "time": "2h ago" }
  ]
}
```

Test Start (Device Lock):

```
{
  "attempt_id": "uuid",
  "questions": [
    {
      "question_id": "uuid",
      "text": "...",
      "options": [{ "id": "A", "text": "..."}, ...],
      "order_index": 1
    }
  ],
  "time_limit_seconds": 1800,
  "deadline": "2026-08-29T18:00:00Z",
  "grace_period_seconds": 21600
}
```

7. Security Architecture

7.1 Authentication & Authorization

Authentication Flow			
1. Teacher: Phone OTP → JWT (30d)			
2. Student: Phone OTP → JWT (90d)			
3. Parent: Token (JWT, 7d, read-only)			
4. Webhook: HMAC signature (Razorpay)			

Role-Based Access Control (RBAC):

Resource	Teacher	Student	Admin
Test CRUD	Own only	Read	All
Attempt	Read	Own only	Read
Notes	Own only	Read	All
Batch	Own only	Read	All
Student	Own only	Read own	All
Challenge	Submit	Submit	Resolve

Settings	Own only	—	Global
----------	----------	---	--------

7.2 Anti-Cheat Implementation

Threat	Mitigation	Implementation
Multi-device attempt	Device fingerprint + Redis lock	ActiveDeviceSession table + Redis device_lock
Tab switching	Client-side detection + progressive warning	JavaScript visibilitychange event → log → flag at 3
App backgrounding	Auto-submit after 60s	Android lifecycle onStop() → timer → auto-submit API
Internet drop	Auto-submit from last saved state	Client queue + server reconciliation
Question leak	Randomized option order per student	Server-side shuffle cached in Redis test_preview
Screenshot sharing	Watermarked full-view images	Server-side overlay: {Name} • {Center}
Answer replay	Unique question set per attempt	Random selection from pool per attempt

7.3 Data Protection

- **Encryption at Rest:** PostgreSQL RDS encryption (AES-256), S3 bucket encryption
- **Encryption in Transit:** TLS 1.3 minimum, certificate pinning in mobile app
- **PII Handling:** Parent emails encrypted at application layer (field-level encryption)
- **Image Security:** Signed URLs with 15-minute expiry, watermark on full-view only
- **Backup Strategy:** Daily automated snapshots (7-day retention), cross-region weekly

8. Infrastructure & Deployment

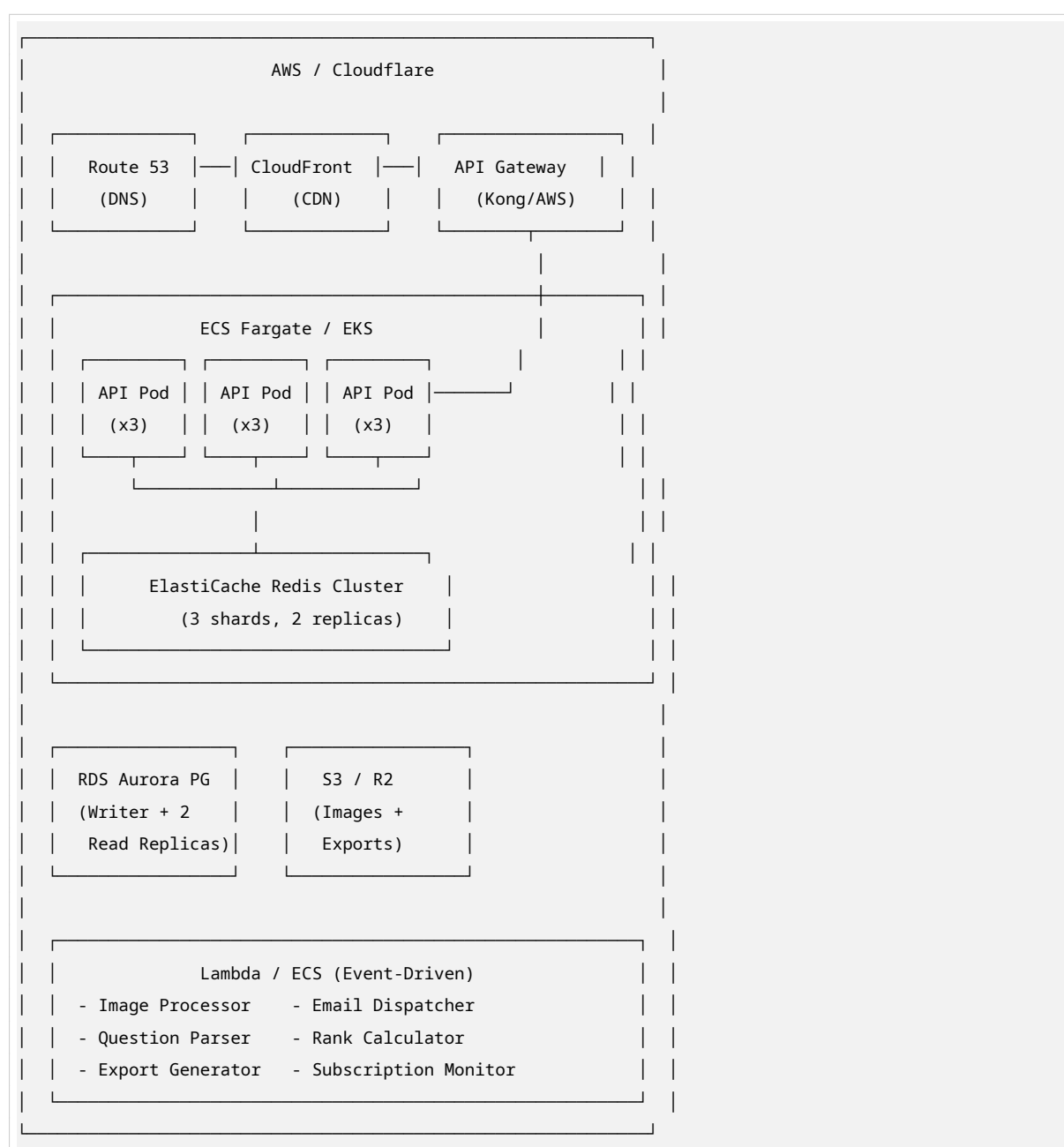
8.1 Recommended Stack (MVP)

Layer	Technology	Rationale
Mobile App	Flutter	Single codebase, Android 8+ support, rich UI for dark theme/glassmorphism
API Framework	Node.js + Fastify	High throughput, low latency, excellent JSON performance
Database	PostgreSQL 15 (RDS Aurora)	ACID compliance, JSON support, mature partitioning
Cache/Queue	Redis 7 (ElastiCache)	Pub/sub for real-time, sorted sets for leaderboards, atomic counters
Object Storage	Cloudflare R2	S3-compatible, zero egress fees (critical for image-heavy notes)
Image Processing	Sharp (Node.js) + Lambda	Thumbnail/watermark generation, async, cost-effective

Table 5 – continued

Layer	Technology	Rationale
Email	Gmail API (OAuth 2.0)	Teacher' s own inbox, lands in Primary, free tier 100/day
Payments	Razorpay	Indian market, UPI support, webhook reliability
Search (v2)	Elasticsearch	Deferred to Phase 2 for note OCR search
Monitoring	Datadog / New Relic	APM, error tracking, custom dashboards
Logging	CloudWatch + S3	Centralized, searchable, cost-optimized

8.2 Deployment Architecture



8.3 Scaling Triggers

Metric	Threshold	Action
API CPU > 70%	2 min	Scale +2 pods
API latency p95 > 500ms	5 min	Scale +2 pods, alert
Redis memory > 80%	1 min	Alert, consider cluster resize
DB connections > 80%	Immediate	Enable read replica routing
Queue depth > 1000	5 min	Scale worker pods
Storage > 80%	24h	Auto-expand S3/R2 (unlimited)

9. Offline & Sync Strategy

9.1 Client-Side Queue

```
// Local Queue Structure (SQLite / Hive)
interface LocalQueueItem {
    id: string;
    attempt_id: string;
    question_id: string;
    selected_option: string;
    timestamp: number; // client clock
    sequence: number; // monotonic counter
    synced: boolean;
}

// Sync Protocol
1. App comes online → check queue
2. POST /api/tests/{id}/sync-local
   Body: { local_queue: LocalQueueItem[], last_server_sequence: number }
3. Server responds: { accepted: Item[], rejected: Item[], server_state: State }
4. Client reconciles: server state wins if conflict, local queue cleared on success
```

9.2 Auto-Save Dual Path

```
Every 30 seconds:
├─ Server save: POST /api/tests/{id}/save → Redis (fast) → Async DB write
└─ Local save: Write to SQLite queue (guaranteed)

On auto-submit (background/timeout):
1. Read latest from local queue
2. Compare timestamp with last server save
3. If local is newer: call /sync-local first
4. Then call /submit with final state
```

10. Tier Enforcement Architecture

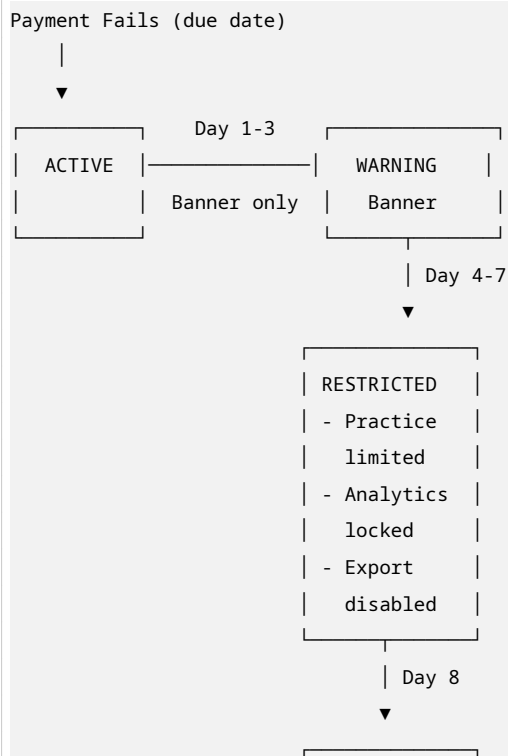
10.1 Feature Gates

```
// Middleware pattern
function requirePremium(feature: PremiumFeature) {
  return async (req, res, next) => {
    const subscription = await getSubscription(req.teacher_id);
    const plan = subscription.plan; // FREE | MONTHLY | ANNUAL
    const status = subscription.status; // TRIAL | ACTIVE | GRACE | SUSPENDED

    if (status === 'SUSPENDED' ||
        (plan === 'FREE' && !freeFeatures.includes(feature))) {
      return res.status(403).json({
        code: 'PREMIUM_REQUIRED',
        feature,
        upgrade_url: '/api/subscription/upgrade'
      });
    }
    next();
  };
}

// Applied to routes
app.post('/api/teacher/weak-topic-test',
  requirePremium('WEAK_TOPIC_TEST'),
  weakTopicTestHandler
);
```

10.2 Grace Period State Machine



	SUSPENDED	
	(Free tier)	

11. Monitoring & Observability

11.1 Key SLIs

SLI	Target	Measurement
Home screen load time	< 1s	Client timing API
Test start API	< 200ms	Server response time
Auto-save latency	< 100ms	Redis write time
Image upload	< 5s	End-to-end (compress + S3)
Email delivery	< 24h	Queue processing time
Uptime	99.9%	Synthetic probes

11.2 Critical Alerts

```
alerts:
- name: device_lock_conflict
  condition: rate(409_responses[5m]) > 10
  severity: warning

- name: auto_submit_failure
  condition: rate(auto_submit_errors[5m]) > 5
  severity: critical

- name: gmail_rate_limit
  condition: rate_limit_usage > 90%
  severity: warning

- name: payment_webhook_failure
  condition: webhook_failure_rate > 1%
  severity: critical

- name: question_pool_exhaustion
  condition: available_questions < 100
  severity: warning
```

12. Disaster Recovery

Scenario	RTO	RPO	Mitigation
AZ failure	< 5 min	0	Multi-AZ RDS, Redis cluster, ECS across AZs
Region failure	< 1 hour	< 5 min	Cross-region read replica promotion

Table 8 – continued

Scenario	RTO	RPO	Mitigation
Data corruption	< 30 min	< 1 hour	Point-in-time recovery (7 days)
Accidental deletion	< 1 hour	< 1 day	S3 versioning, RDS snapshots
DDoS attack	< 2 min	0	Cloudflare/CDN, WAF, rate limiting

13. Phase 2 Evolution Path

Current (MVP)	Phase 2 Enhancement
Single API monolith	Service split (Test Engine, Content, Payment as separate services)
PostgreSQL only	Add Elasticsearch for note/question search
In-app notifications only	Add FCM push notifications
Android only	iOS + Web dashboard (React/Vue)
Manual question upload	API integration with external question banks
Single-teacher	Multi-teacher role management
English only	Regional language support (i18n)
CBSE only	State board support
Token-based parent view	Full parent app with login
Device ID deferred	Robust device fingerprinting strategy

14. Decision Log (Architecture)

Decision	Rationale	Trade-off
Node.js over Python/Go	Team familiarity, JSON-heavy API, fast iteration	Slightly lower CPU efficiency vs. Go
PostgreSQL over MongoDB	ACID for financial data, complex joins for analytics, JSONB for flexibility	Schema migration overhead
Redis over Kafka for events	Simpler ops, sufficient for MVP volume, built-in data structures	Less durable for critical events (compensate with DB)
Cloudflare R2 over AWS S3	Zero egress fees critical for image-heavy notes	Slightly higher latency in India
Gmail API over SES/SendGrid	Lands in Primary inbox, teacher's own identity, free	100/day limit requires queuing
Flutter over React Native	Better performance for image galleries, single language (Dart)	Larger app size
Monolith over microservices	Faster MVP delivery, simpler debugging, lower infra cost	Will split at Phase 2

Document prepared for development kickoff. Review required before implementation begins.